

OOSE Mod 4

Modeling and Design

Principles that Guide Process

Principle 1. Be agile. Whether the process model you choose is prescriptive or agile, the basic tenets of agile development should govern your approach. Every aspect of the work you do should emphasize economy of action—keep your technical approach as simple as possible, keep the work products you produce as concise as possible, and make decisions locally whenever possible.

Principle 2. Focus on quality at every step. The exit condition for every process activity, action, and task should focus on the quality of the work product that has been produced.

Principle 3. Be ready to adapt. Process is not a religious experience, and dogma has no place in it. When necessary, adapt your approach to constraints imposed by the problem, the people, and the project itself.

Principle 4. Build an effective team. Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team that has mutual trust and respect.

Principle 5. Establish mechanisms for communication and coordination. Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts to create a successful end product. These are management issues and they must be addressed.

Principle 6. Manage change. The approach may be either formal or informal, but mechanisms must be established to manage the way changes are requested, assessed, approved, and implemented.

Principle 7. Assess risk. Lots of things can go wrong as software is being developed. It's essential that you establish contingency plans.

Principle 8. Create work products that provide value for others. Create only those work products that provide value for other process activities, actions, or tasks. Every work product that is produced as part of software engineering practice will be passed on to someone else. A list of required functions and features will be passed along to the person (people) who will develop a design, the design will be passed along to those who generate code, and so on. Be sure that the work product imparts the necessary information without ambiguity or omission

Principles that Guide Practice

Principle 1: Divide and Conquer: Break large problems into smaller, focused parts using Separation of Concerns (SoC). Each part should handle a distinct responsibility and be developed or tested independently, making the overall system easier to manage.

Principle 2: Understand the Use of Abstraction: Use abstraction to simplify complexity by

focusing on essential features and hiding the rest. While abstraction helps communicate ideas clearly, all non-trivial abstractions may "leak," revealing hidden complexities—so it's important to understand both high-level concepts and underlying details.

Principle 3: Strive for Consistency: Maintain consistency in design, coding, and interface layout. Familiar patterns—like consistent menu placement or color schemes—make systems easier to understand, use, and maintain.

Principle 4: Focus on the Transfer of Information: Software revolves around moving information between components or users. Design and test interfaces carefully to ensure accurate, clear, and reliable information flow across all parts of the system.

Principle 5: Build Software That Exhibits Effective Modularity: Modularity brings SoC into practice. Good modules should focus on one function (high cohesion) and have minimal dependency on others (low coupling), enabling easier development, testing, and maintenance.

Principle 6: Look for Patterns: Use proven design patterns to solve recurring problems. Patterns form a shared language and knowledge base that helps developers build well-structured, reliable, and maintainable software.

Principle 7: Represent the Problem and Solution from Multiple Perspectives: View the system using different angles—like data flow, functionality, and behavior—to gain deeper understanding and uncover errors or gaps in requirements or design.

Principle 8: Remember That Someone Will Maintain the Software: Design with future maintenance in mind. Software will evolve over time due to bugs, user needs, or environmental changes. Writing clean, well-structured code and documentation ensures others can maintain and improve it easily.

Why Modeling is Important in Software Engineering

Modeling provides a structured, simplified view of complex software systems, making it easier for stakeholders to understand, analyze, and communicate key aspects throughout development.

- Simplifies Complexity
 - Breaks down systems into manageable parts, helping in analysis and problem-solving.
- Improves Communication
 - Acts as a common language between technical and non-technical stakeholders.
- Enhances Design Quality
 - Helps identify flaws early and ensures consistency and completeness.
- Enables Early Validation
 - Allows testing of ideas and requirements before coding begins.
- Supports Reuse & Scalability
 - Encourages modular, reusable designs that adapt to future growth.
- Improves Documentation
 - Creates clear records for maintenance and onboarding.
- Enables Automation
 - Facilitates code generation, testing, and simulation from models.
- Reduces Development Risks
 - Uncovers issues early, preventing costly late-stage errors.

Unified Modeling Language (UML):

It is a standard visual language for describing and modelling software blueprints. UML is more than just a graphical language. Stated formally, the UML is for: Visualizing, Specifying, Constructing, and Documenting. The artifacts of a software-intensive system (particularly systems built using the object-oriented style).

The conceptual model of UML contains the fundamentals of UML. The conceptual model consists of three parts. They are:

- 1) Building blocks of UML (syntax / vocabulary)
- 2) Rules (semantics)
- 3) Common Mechanisms

Building blocks of UML (Conceptual model of UML): These are the fundamental elements in UML. Every diagram can be represented using these building blocks. The building blocks of UML contain three types of elements. They are:

- Things (object oriented parts of uml)
- Relationships (relational parts of uml)
- Diagrams

Things: A diagram can be viewed as a graph containing vertices and edges. In UML, vertices are replaced by things, and the edges are replaced by relationships. There are four types of things in UML. They are:

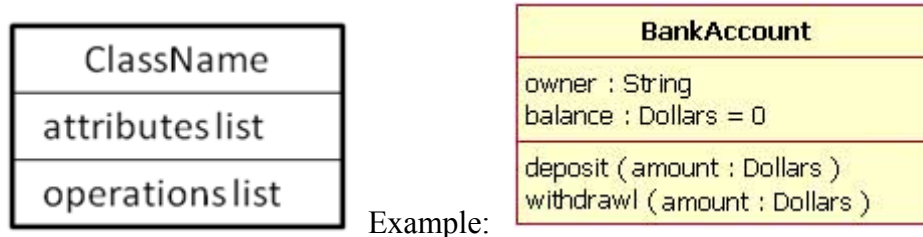
- Structural things (nouns of uml – static parts)
- Behavioral things (verbs of uml – dynamic parts)
- Grouping things (organizational parts)

- Annotational things (explanatory parts)

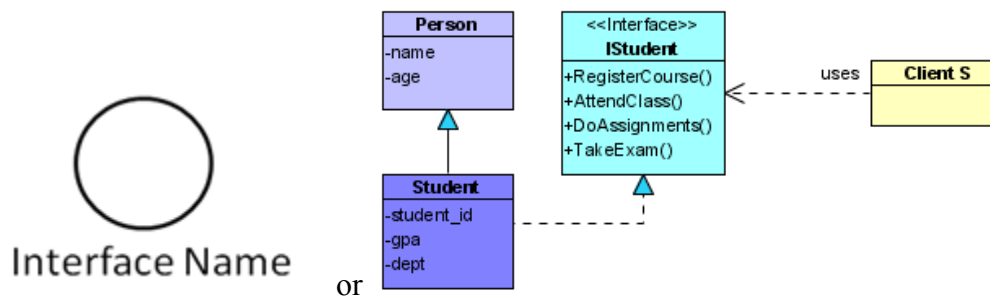
Structural things:

Represents the static aspects of a software system. There are seven structural things in UML. They are:

Class: A class is a collection of similar objects having similar attributes, behavior, relationships and semantics. Graphically the class is represented as a rectangle with three compartments.



Interface: An interface is a collection of operation signatures and/or attribute definitions that ideally define a cohesive set of behavior. Graphically interface is represented as a circle or a class symbol stereotyped with interface.



Use Case: A use case is a collection of actions, defining the interactions between a role (actor) and the system. Graphically the use case is represented as a solid ellipse with its name written inside or below the ellipse.



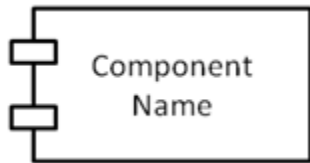
Collaboration: A collaboration is the collection of interactions among objects to achieve a goal. Graphically collaboration is represented as a dashed ellipse. A collaboration can be a collection of classes or other elements.



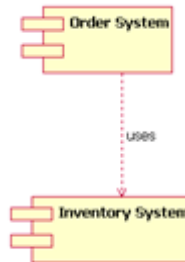
Example:



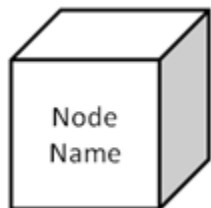
Component: A component is a physical and replaceable part of a system. Graphically Component is represented as a tabbed rectangle. Examples of components are executable files, dll files, database tables, files and documents.



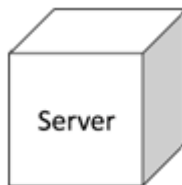
Example:



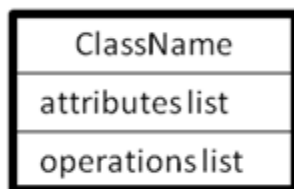
Node: A node is a physical element that exists at run time and represents a computational resource. Graphically node is represented as a cube. Examples of nodes are PCs, laptops, smartphones or any embedded system.



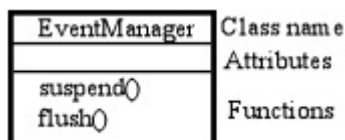
Example:



Active Class: A class whose objects can initiate its own flow of control (threads) and work in parallel with other objects. Graphically active class is represented as a rectangle with thick borders.



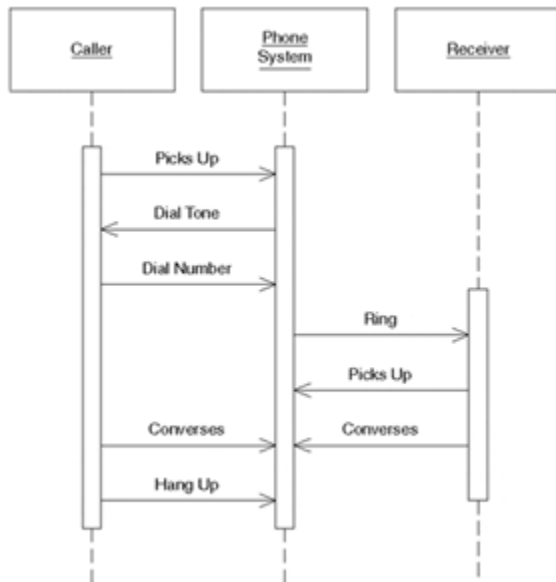
Example:



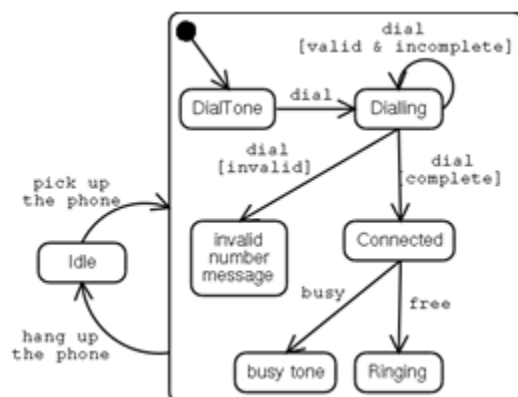
Behavioral Things:

Represents the dynamic aspects of a software system. Behavior of a software system can be modeled as interactions or as a sequence of state changes.

Interaction: A behavior made up of a set of messages exchanged among a set of objects to perform a particular task. A message is represented as a solid arrow. Below is an example of interaction representing a phone conversation:



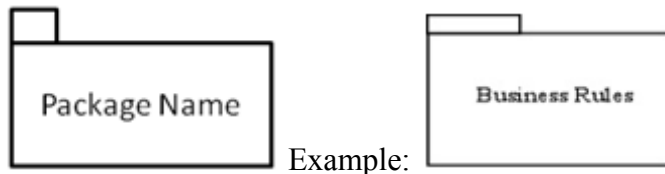
State Machine: A behavior that specifies the sequences of states an object or interaction goes through during its lifetime in response to events. A state is represented as a rectangle with rounded corners. Below is an example of state machine representing the states of a phone system:



Grouping Things

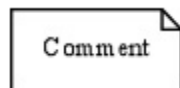
Elements which are used for organizing related things and relationships in models.

Package: A general purpose mechanism for organizing elements into groups. The Graphical package is represented as a tabbed folder. When the diagrams become large and cluttered, relatives are grouped into a package so that the diagram can become less complex and easy to understand.



Annotational Things

A symbol to display comments. Graphically note is represented as a rectangle with a dog ear at the top right corner.



Relationships:

The things in a diagram are connected through relationships. So, a relationship is a connection between two or more things.

Dependency: A semantic relationship, in which a change in one thing (the independent thing) may cause changes in the other thing (the dependent thing). This relationship is also known as “using” relationship. Graphically represented as a dashed line with a stick arrow head.



Example:



Association: A structural relationship describing connections between two or more things. Graphically represented as a solid line with optional stick arrow representing navigation.



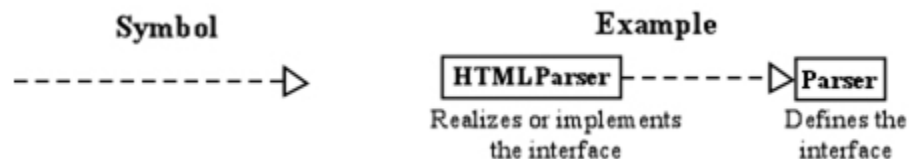
Generalization: Is a generalization-specialization relationship. Simply put this describes the relationship of a parent class (generalization) to its subclasses (specializations). Also known as “is-a” relationship.



Example:



Realization: Defines a semantic relationship in which one class specifies something that another class will perform. Example: The relationship between an interface and the class that realizes or executes that interface.



Diagrams

A diagram is a collection of elements often represented as a graph consisting of vertices and edges joining these vertices. These vertices in UML are things and the edges are relationships. UML includes nine diagrams:

- Class diagram
- Object diagram
- Use case diagram
- Component diagram
- Deployment diagram
- Sequence diagram
- Collaboration diagram
- Statechart diagram and
- Activity diagram.

Rules (conceptual model of UML)

The rules of UML specify how the UMLs building blocks come together to develop diagrams. The rules enable the users to create well-formed models. A well-formed model is self-consistent and also consistent with the other models. UML has rules for:

- Names – What elements can be called as things, relationships and diagrams
- Scope – The context that gives a specific meaning to a name

- Visibility – How these names are seen and can be used by the other names
- Integrity – How things properly relate to one another
- Execution – What it means to run or simulate a model

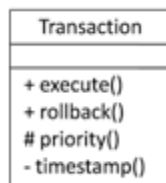
Common Mechanisms in UML (conceptual model of uml)

Why is UML easy to learn and use? It's because of the four common mechanisms that apply throughout the UML. They are:

1. Specifications
2. Adornments
3. Common divisions
4. Extensibility mechanisms

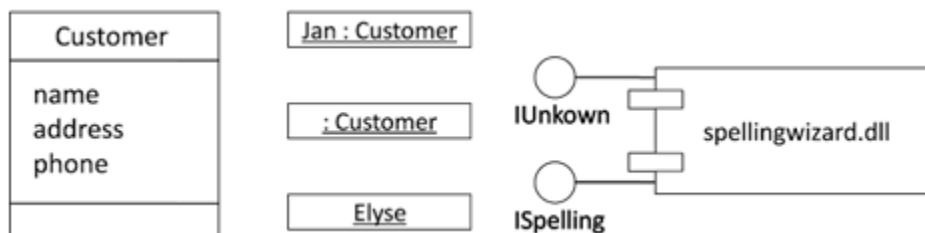
Specifications: Behind every graphical notation in UML there is a precise specification of the details that element represents. For example, a class icon is a rectangle and it specifies the name, attributes and operations of the class.

Adornments: The mechanism in UML which allows the users to specify extra information with the basic notation of an element is the adornments.



In the above example, the access specifiers: + (public), # (protected) and – (private) represent the visibility of the attributes which is extra information over the basic attribute representation.

Common Divisions: In UML there is clear division between semantically related elements like: separation between a class and an object and the separation between an interface and its implementation.

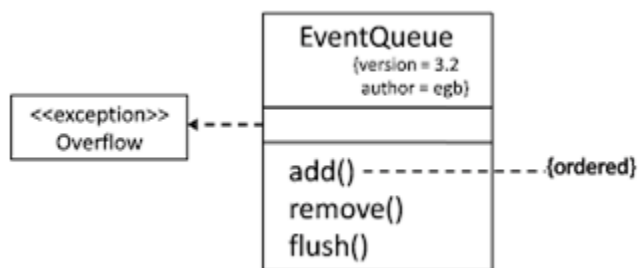


Extensibility Mechanisms

UMLs extensibility mechanisms allow the user to extend (new additions) the language in a controlled way. The extensibility mechanisms in UML are:

- Stereotypes – Extends the vocabulary of UML. Allows users to declare new building blocks (icons) or extend the basic notations of the existing building blocks by stereotyping them using guillemets.
- Tagged Values – Extends the properties of an UML building block. Allows us to specify extra information in the elements specification. Represented as text written inside braces and placed under the element name. The general syntax of a property is:
{ property name = value }
- Constraints – Extends the semantics of a UMLs building block such as specifying new rules or modifying existing rules. Represented as text enclosed in braces and placed adjacent or beside the element name.

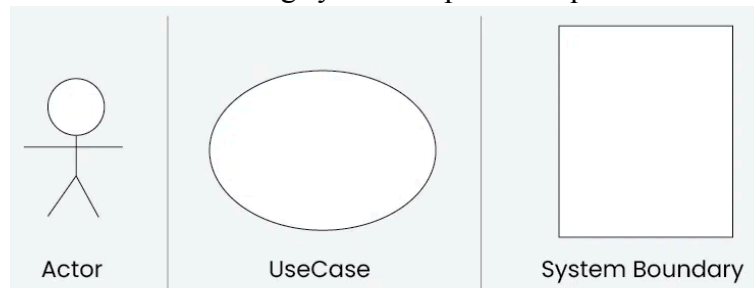
Example:



In the above example, we are specifying the exception “Overflow” using the class symbol and stereotyping it with “exception”. Also under the class name, “EventQueue” we are specifying additional properties like “version” and “author” using tagged values. Finally, we are constraining the operation named “add” that before adding a new event to the EventQueue object, all the events must be “ordered” in some manner. This is specified using constraints in UML.

Use Case Diagram:

A Use Case Diagram in Unified Modeling Language (UML) is a visual representation that illustrates the interactions between users (actors) and a system. It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system. Use case diagrams provide a high-level overview of a system's behavior, making them useful for stakeholders, developers, and analysts to understand how a system is intended to operate from the user's perspective, and how different processes relate to one another. They are crucial for defining system scope and requirements.



When to apply Use Case Diagrams?

Use case diagrams are useful in several situations. Here's when you should consider using them:

- When you need to gather and clarify user requirements, use case diagrams help visualize how different users interact with the system.
- If you're working with diverse groups, including non-technical stakeholders, these diagrams provide a clear and simple way to convey system functionality.
- During the system design phase, use case diagrams help outline user interactions and plan features, ensuring that the design aligns with user needs.
- When defining what is included in the system versus what is external, use case diagrams help clarify these boundaries.

Use Case Diagram Notations

UML notations provide a visual language that enables software developers, designers, and other stakeholders to communicate and document system designs, architectures, and behaviors in a consistent and understandable manner.

1. **Actors:** Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.
2. **Use Cases:** Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals.
3. **System Boundary:** The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

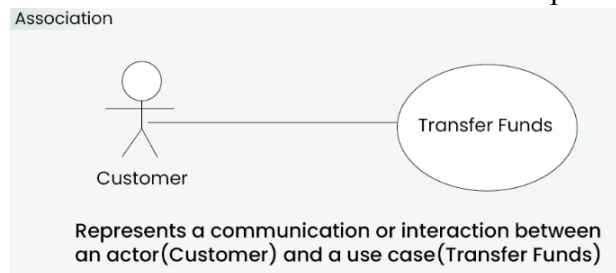
Use Case Diagram Relationships: In a Use Case Diagram, relationships play a crucial role in depicting the interactions between actors and use cases. These relationships provide a comprehensive view of the system's functionality and its various scenarios. Let's delve into the key types of relationships and explore examples to illustrate their usage.

Association Relationship

The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- Actor: Customer
- Use Case: Transfer Funds
- Association: A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

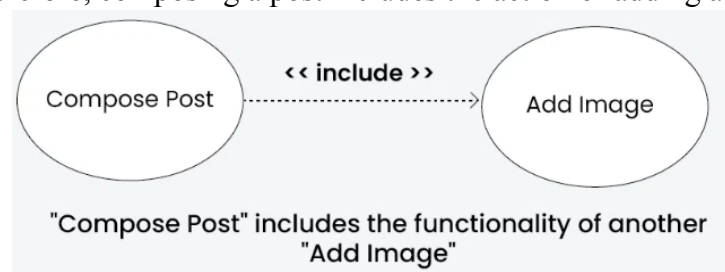


Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

Example: Social Media Posting

- Use Cases: Compose Post, Add Image
- Include Relationship: The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.

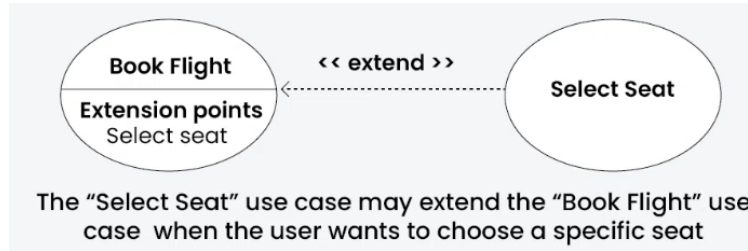


Extend Relationship

The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword "extend." This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

- Use Cases: Book Flight, Select Seat
- Extend Relationship: The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.

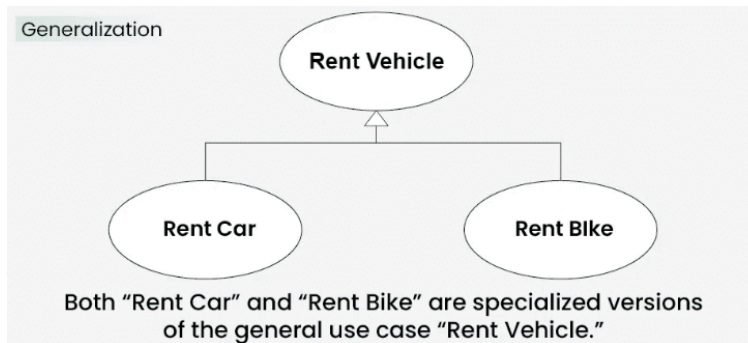


Generalization Relationship

The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

- Use Cases: Rent Car, Rent Bike
- Generalization Relationship: Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."



How to draw a Use Case diagram in UML?

- Step 1: Identify Actors: Determine who or what interacts with the system. These are your actors. They can be users, other systems, or external entities.
- Step 2: Identify Use Cases: Identify the main functionalities or actions the system must perform. These are your use cases. Each use case should represent a specific piece of functionality.
- Step 3: Connect Actors and Use Cases: Draw lines (associations) between actors and the use cases they are involved in. This represents the interactions between actors and the system.
- Step 4: Add System Boundary: Draw a box around the actors and use cases to represent the system boundary. This defines the scope of your system.
- Step 5: Define Relationships: If certain use cases are related or if one use case is an extension of another, you can indicate these relationships with appropriate notations.
- Step 6: Review and Refine: Step back and review your diagram. Ensure that it accurately represents the interactions and relationships in your system. Refine as needed.
- Step 7: Validate: Share your use case diagram with stakeholders and gather feedback. Ensure that it aligns with their understanding of the system's functionality.

Use Case Diagram example(Online Shopping System)

Actors:

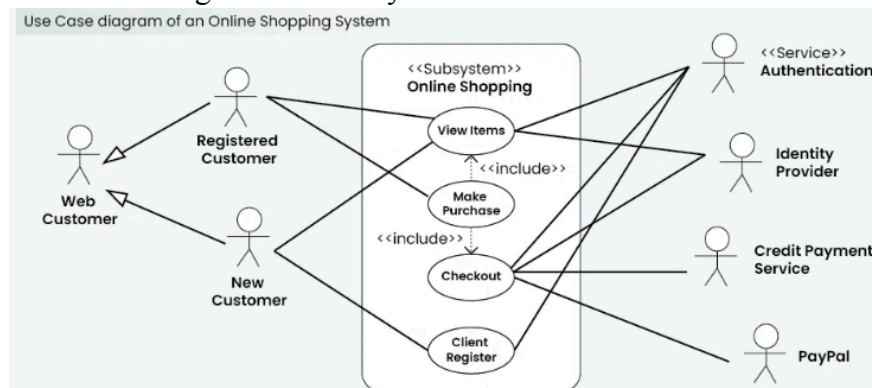
- Customer
- Admin

Use Cases:

- Browse Products
- Add to Cart
- Checkout
- Manage Inventory (Admin)

Relations:

- The Customer can browse products, add to the cart, and complete the checkout.
- The Admin can manage the inventory.



What is the Purpose and Benefits of Use Case Diagrams?

The Use Case Diagram offers numerous benefits throughout the system development process.

Here are some key advantages of using Use Case Diagrams:

- Use Case Diagrams offer a clear visual representation of a system's functions and its interactions with external users. This representation helps stakeholders, including those without technical expertise, in grasping the system's overall behavior.
- They establish a shared language for articulating system requirements, ensuring that all team members have a common understanding.
- Use Case Diagrams illustrate the different ways users engage with the system, contributing to a thorough comprehension of its functionalities.
- In the design phase, Use Case Diagrams help outline how users (actors) will interact with the system. They support the planning of user interfaces and aid in structuring system functionalities.

Activity Diagrams:

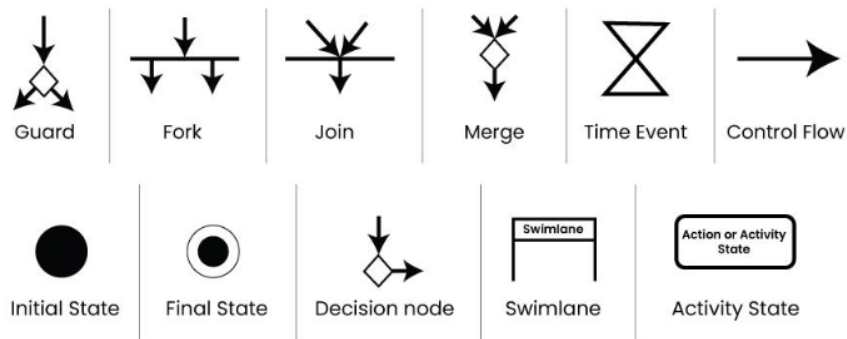
Activity diagrams show the steps involved in how a system works, helping us understand the flow of control. They display the order in which activities happen and whether they occur one after the other (sequential) or at the same time (concurrent). These diagrams help explain what triggers certain actions or events in a system.

An activity diagram starts from an initial point and ends at a final point, showing different decision paths along the way. They are often used in business and process modeling to show how a system behaves over time.

When to use an Activity Diagram?

- **Modeling Workflows or Processes:** When you need to map out a business process, workflow, or the steps involved in a use case, activity diagrams help visualize the flow of activities.
- **Concurrent or Parallel Processing:** If your system or process involves activities happening simultaneously, an activity diagram can clearly show the parallel flow of tasks.
- **Understanding the Dynamic Behavior:** When it's essential to depict how a system changes over time and moves between different states based on events or conditions, activity diagrams are effective.
- **Clarifying Complex Logic:** Use an activity diagram to simplify complex decision-making processes with branching paths and different outcomes.
- **System Design and Analysis:** During the design phase of a software system, activity diagrams help developers and stakeholders understand how different parts of the system interact dynamically.
- **Describing Use Cases:** They are useful for illustrating the flow of control within a use case, showing how various components of the system interact during its execution.

Activity Diagram Notations



Initial State: The starting state before an activity takes place is depicted using the initial state. A process can have only one initial state unless we are depicting nested activities. We use a black filled circle to depict the initial state of a system. For objects, this is the state when they are instantiated. The Initial State from the UML Activity Diagram marks the entry point and the initial Activity State.

Action or Activity State: An activity represents execution of an action on objects or by objects. We represent an activity using a rectangle with rounded corners. Basically any action or event that takes place is represented using an activity.

Action Flow or Control flows: Action flows or Control flows are also referred to as paths and edges. They are used to show the transition from one activity state to another activity state. An activity state can have multiple incoming and outgoing action flows. We use a line with an arrow head to depict a Control Flow. If there is a constraint to be adhered to while making the transition it is mentioned on the arrow.

Decision node and Branching: When we need to make a decision before deciding the flow of

control, we use the decision node. The outgoing arrows from the decision node can be labelled with conditions or guard expressions. It always includes two or more output arrows.

Guard: A Guard refers to a statement written next to a decision node on an arrow sometimes within square brackets. The statement must be true for the control to shift along a particular direction. Guards help us know the constraints and conditions which determine the flow of a process.

Fork: Fork nodes are used to support concurrent activities. When we use a fork node when both the activities get executed concurrently i.e. no decision is made before splitting the activity into two parts. Both parts need to be executed in case of a fork statement. We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent activity state and outgoing arrows towards the newly created activities.

Join: Join nodes are used to support concurrent activities converging into one. For join notations we have two or more incoming edges and one outgoing edge.

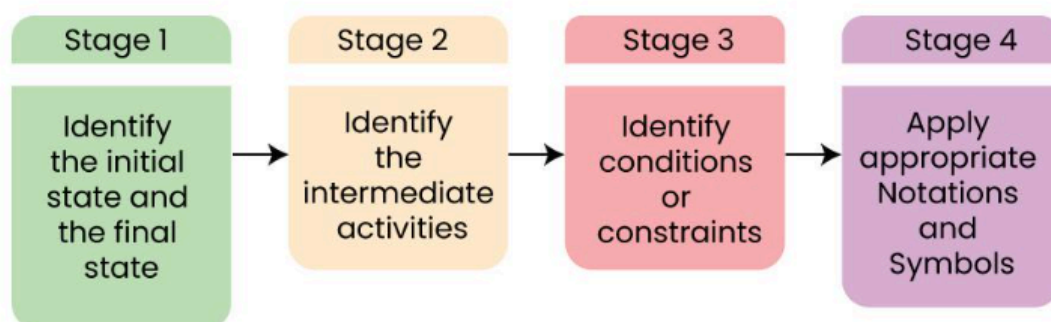
Merge or Merge Event: Scenarios arise when activities which are not being executed concurrently have to be merged. We use the merge notation for such scenarios. We can merge two or more activities into one if the control proceeds onto the next activity irrespective of the path chosen.

Swimlanes: We use Swimlanes for grouping related activities in one column. Swimlanes group related activities into one column or one row. Swimlanes can be vertical and horizontal. Swimlanes are used to add modularity to the activity diagram. It is not mandatory to use swimlanes. They usually give more clarity to the activity diagram. It's similar to creating a function in a program. It's not mandatory to do so, but it is a recommended practice.

Time Event: This refers to an event that stops the flow for a time; an hourglass depicts it. We can have a scenario where an event takes some time to complete.

Final State or End State: The state which the system reaches when a particular process or activity ends is known as a Final State or End State. We use a filled circle within a circle notation to represent the final state in a state machine diagram. A system or a process can have multiple final states.

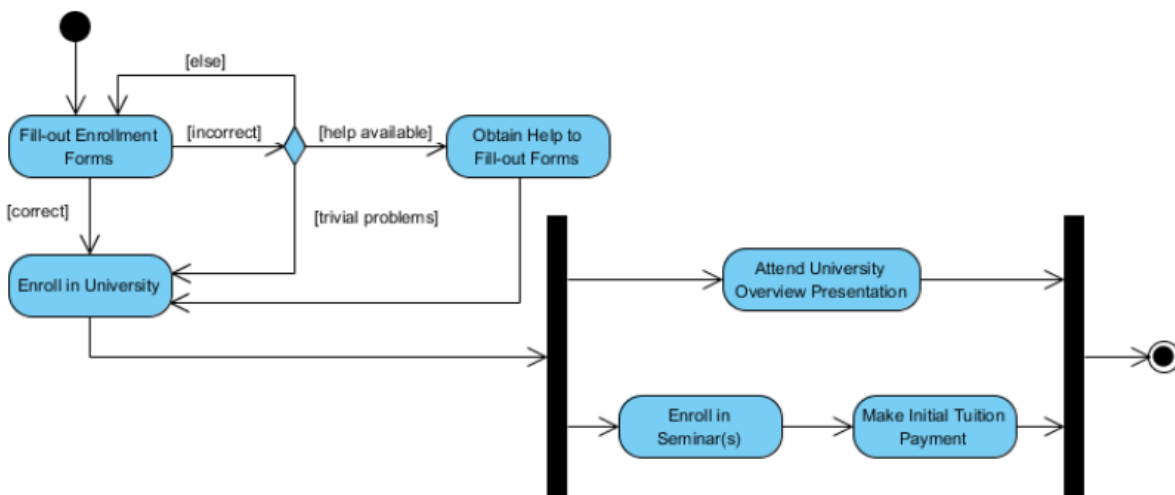
Steps to draw:



Activity Diagram Example - Student Enrollment

This UML activity diagram example describes a process for student enrollment in a university as follows:

- An applicant wants to enroll in the university.
- The applicant hands a filled out copy of Enrollment Form.
- The registrar inspects the forms.
- The registrar determines that the forms have been filled out properly.
- The registrar informs student to attend in university overview presentation.
- The registrar helps the student to enroll in seminars
- The registrar asks the student to pay for the initial tuition.



More examples on visual paradigm site

State Diagrams:

A State diagram is a UML diagram which is used to represent the condition of the system or part of the system at finite instances of time. It's a behavioral diagram and it represents the behavior using finite state transitions.

- State Machine diagrams are also known as State Diagrams and State-Chart Diagrams. These both terms can be used interchangeably.
- A state machine diagram is used to model the dynamic behaviour of a class in response to time and changing external stimuli(events that cause the system to change its state from one to another).
- We can say that every class has a state but we don't model every class using State Machine diagrams.

Basic components of a statechart diagram:

Initial state – We use a black filled circle to represent it in a System or a class.

Transition – We use a solid arrow to represent the transition or change of control from one state to another. The arrow is labelled with the event which causes the change in state.

State – We use a rounded rectangle to represent a state. A state represents the conditions or circumstances of an object of a class at an instant of time.

Fork – We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent state and outgoing arrows towards the newly created states. We use the fork notation to represent a state splitting into two or more concurrent states.

Join – We use a rounded solid rectangular bar to represent a Join notation with incoming arrows from the joining states and outgoing arrow towards the common goal state. We use the join notation when two or more states concurrently converge into one on the occurrence of an event or events.

Self transition – We use a solid arrow pointing back to the state itself to represent a self transition. There might be scenarios when the state of the object does not change upon the occurrence of an event. We use self transitions to represent such cases.

Composite state – We use a rounded rectangle to represent a composite state also. We represent a state with internal activities using a composite state.

Final state – We use a filled circle within a circle notation to represent the final state in a state machine diagram.

Steps to draw a state diagram –

1. Identify the initial state and the final terminating states.
2. Identify the possible states in which the object can exist (boundary values corresponding to different attributes guide us in identifying different states).
3. Label the events which trigger these transitions.

Example – state diagram for an online order –

1. In the event of an order being received, we transit from our initial state to the Unprocessed order state.
2. The unprocessed order is then checked.
3. If the order is rejected, we transit to the Rejected Order state.
4. If the order is accepted and we have the items available we transit to the fulfilled order state.
5. However if the items are not available we transit to the Pending Order state.
6. After the order is fulfilled, we transit to the final state. In this example, we merge the two states i.e. Fulfilled order and Rejected order into one final state.

Note – Here we could have also treated fulfilled order and rejected order as final states separately.

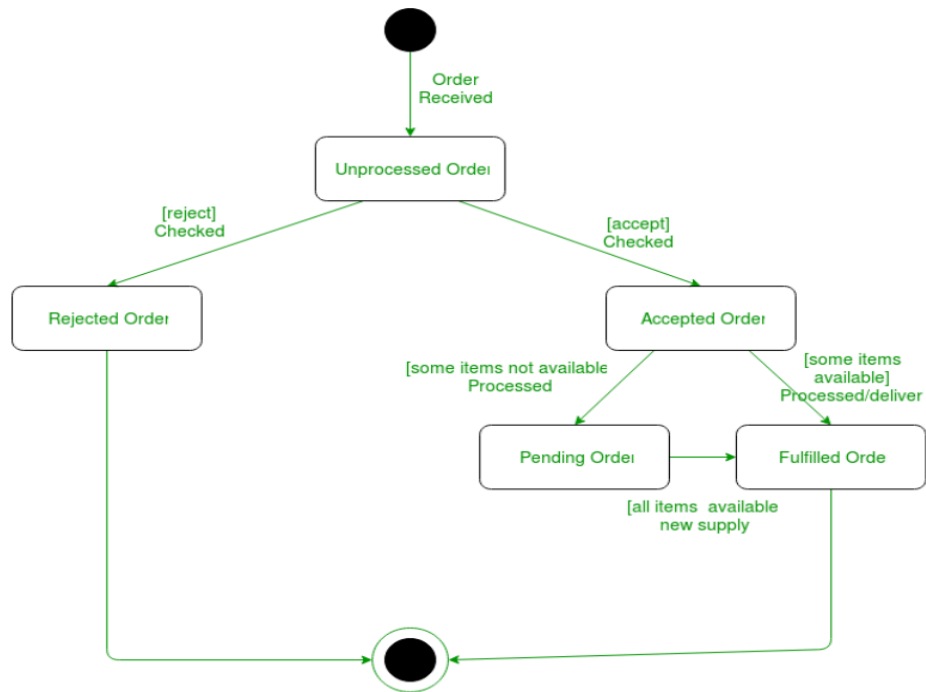


Figure – state diagram for an online order

Interaction Diagram:

In UML (Unified Modeling Language), interaction diagrams are a category of diagrams used to visualize and describe the dynamic aspects of a system or software application. These diagrams focus on illustrating how different objects or components within the system interact with each other over time. Interaction diagrams are particularly useful for modeling and understanding the flow of messages, actions, and events that occur during the execution of a use case or scenario.

Interaction diagrams are essential tools in UML for capturing and communicating the dynamic behavior of a system. They help software developers, designers, and stakeholders understand how different parts of a system interact and communicate during various scenarios, aiding in the design, analysis, and communication of system behavior.

Types of Interaction Diagrams

- **Sequence Diagrams:** Sequence diagrams depict interactions between objects or components in a time-ordered sequence. They are often used to model the behavior of a single use case or scenario. In a sequence diagram, you represent objects as lifelines (vertical lines) and show the messages exchanged between these lifelines over time. This diagram helps in visualizing the chronological order of interactions and the messages exchanged between objects.
- **Communication Diagrams:** Communication diagrams, also known as collaboration diagrams, provide a different perspective on object interactions. They emphasize the structural relationships between objects and how they collaborate to achieve certain behaviors or scenarios. In a communication diagram, you depict objects as nodes, and connecting lines represent the interactions or messages exchanged between them. These diagrams are useful for understanding the relationships between objects in a system.

Two Other Interaction Diagram Variants

- **Interaction Overview Diagrams** : Interaction Overview Diagrams define interactions in a manner that emphasizes an overview of the control flow. These diagrams incorporate notational elements similar to those found in Activity diagrams, such as flow lines, forks, joins, and more. However, it is essential to note that while the notation and general purpose of these elements resemble those in Activity diagrams, their detailed semantics differ significantly. Consequently, modelers should avoid interpreting Interaction Overview Diagrams as if they were Activity diagrams.
- **Timing Diagrams**: Timing Diagrams serve as a means to illustrate interactions when the primary objective is to analyze time-related aspects.

Interaction Diagram Type	Purpose and Focus	Key Elements
Sequence Diagrams	Depict time-ordered interactions between objects or components.	Lifelines (vertical lines representing objects or components), messages, and their ordering.
Communication Diagrams	Illustrate structural relationships between objects and their interactions.	Nodes (representing objects or components), connecting lines (for interactions), and object relationships.
Interaction Overview Diagrams	Provide an overview of control flow in interactions.	Notational elements (e.g., flow lines, forks, joins), similar to Activity diagrams but with different semantics.
Timing Diagrams	Focus on interactions where timing is crucial.	Time intervals, lifelines, events, and messages with a time dimension.

Interaction Sequence Diagrams:

Interaction sequence diagrams are a type of UML (Unified Modeling Language) interaction diagram that visually models the dynamic behavior of a system by showing how objects interact with each other through the exchange of messages over time. They are particularly useful for illustrating the flow of events in a specific scenario or use case, focusing on the time-ordering of interactions between system components.

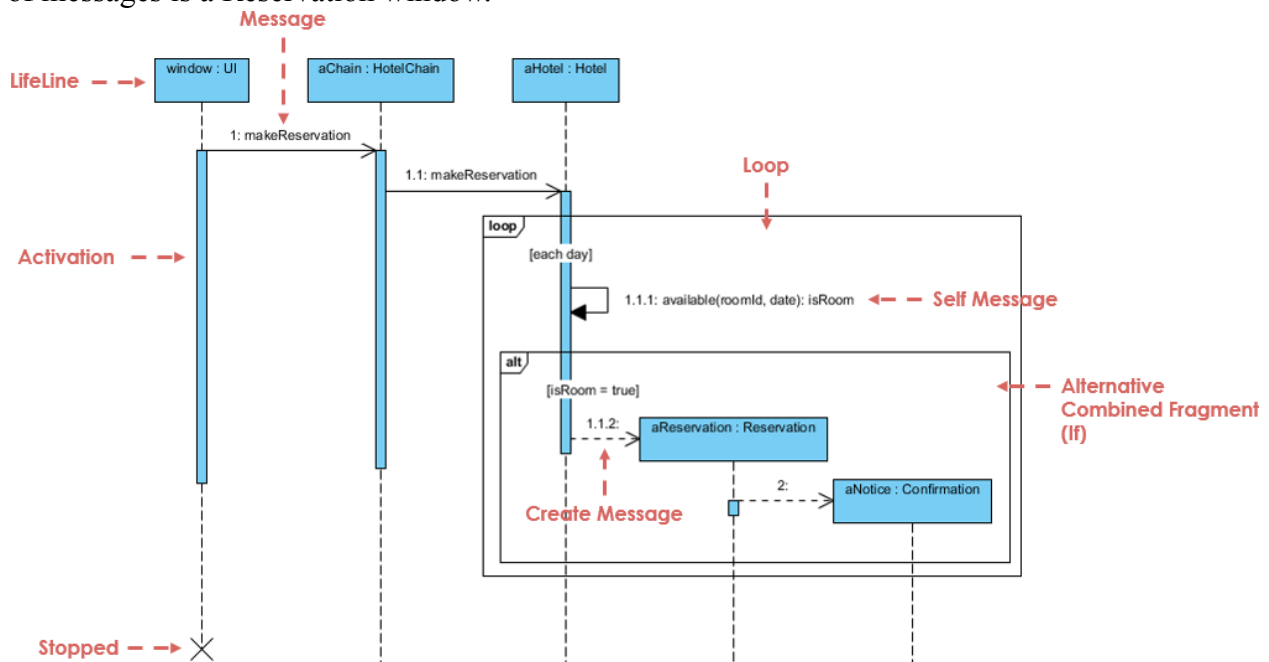
- **Lifelines**: Represent individual objects or participants involved in the interaction, depicted as vertical dashed lines.
- **Messages**: Horizontal arrows between lifelines that show communication or method calls, arranged from top to bottom to indicate the sequence in which they occur.
- **Time Axis**: The vertical axis represents the progression of time, with earlier messages at the top and later ones at the bottom.
- **Fragments**: Special constructs (like loops, alternatives, or parallel executions) can be used to represent complex control flows.

How Sequence Diagrams Are Made: Steps to Create a Sequence Diagram:

- **Identify the Scenario**: Determine the specific scenario or use case you want to model. This could be a business process, a system function, or a user interaction.

- List the Participants: Identify all objects, actors, or components (lifelines) that participate in the interaction. Place these lifelines horizontally across the top of the diagram.
- Map the Sequence of Messages: For each step in the scenario, draw a horizontal arrow from the sender's lifeline to the receiver's lifeline, labeling it with the message or method name. Arrange messages from top (earliest) to bottom (latest) to reflect the order of operations.
- Add Control Structures (Optional)
- Use fragments such as:
 - alt: Alternatives (if-else branches)
 - opt: Optional behavior (if condition is true)
 - loop: Repeated interactions
 - par: Parallel processes
- Annotate as Needed: Add notes, comments, or constraints to clarify aspects of the interaction.
- Review and Refine: Ensure the diagram accurately represents the intended behavior and is easy to understand for stakeholders.

Below is a sequence diagram for making a hotel reservation. The object initiating the sequence of messages is a Reservation window.



Communication Diagram:

UML communication diagrams, like the sequence diagrams - a kind of interaction diagram, shows how objects interact. A communication diagram is an extension of object diagram that shows the objects along with the messages that travel from one to another. In addition to the associations among objects, a communication diagram shows the messages the objects send each other.

Purpose of Communication Diagram

- Model message passing between objects or roles that deliver the functionalities of use cases and operations
- Model mechanisms within the architectural design of the system
- Capture interactions that show the passed messages between objects and roles within the collaboration scenario
- Model alternative scenarios within use cases or operations that involve the collaboration of different objects and interactions
- Support the identification of objects (hence classes), and their attributes (parameters of message) and operations (messages) that participate in use cases

Communication Diagram Elements

Objects participating in a collaboration come in two flavors: supplier and client.

- Supplier objects are the objects that supply the method that is being called, and therefore receive the message.
- Client objects call methods on supplier objects, and therefore send messages.

Links

- The connecting lines drawn between objects in a communication diagram are links.
- These links are what set communication diagrams apart from sequence diagrams. They enable you to see the relationships between objects.
- Each link represents a relationship between objects and symbolizes the ability of objects to send messages to each other.
- If an object sends messages to itself, the link carrying these messages is represented as a loop icon. This loop can be seen on both the UI object and the Transaction object.

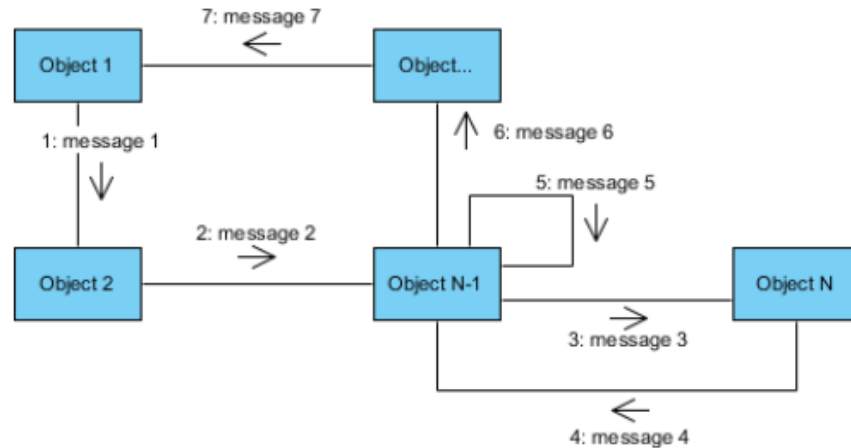
Messages in communication diagrams are shown as arrows pointing from the Client object to the Supplier object. Typically, messages represent a client invoking an operation on a supplier object. They can be modeled along with the objects in the following manner:

- Message icons have one or more messages associated with them.
- Messages are composed of message text prefixed by a sequence number.
- This sequence number indicates the time-ordering of the message.

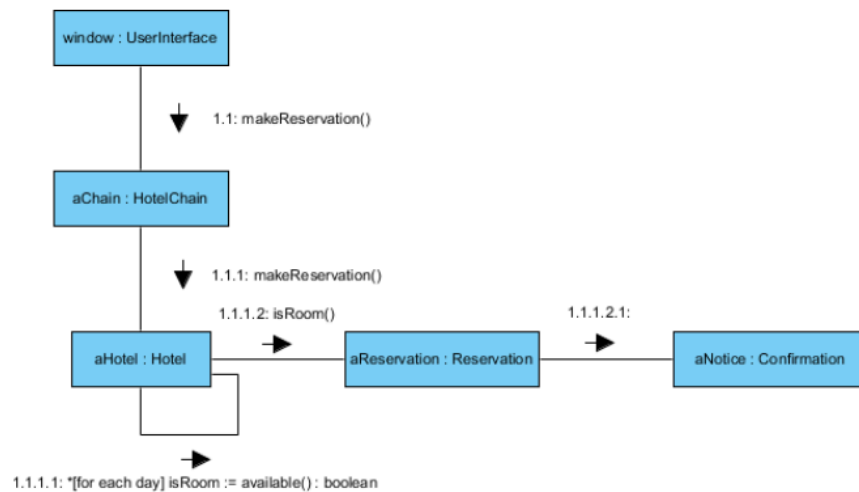
Communication Diagram at a Glance

In the example of the notation for a communication diagram, objects (actors in use cases) are represented by rectangles. In the example (generic communication diagram):

- The objects are Object1, Object2, Object..., ObjectN-1 ..., and ObjectN.
- Messages passed between objects are represented by labeled arrows that start with the sending object (actor) and end with the receiving object.
- The sample messages passed between objects are labeled 1: message1, 2: message2, 3: message3, etc., where the numerical prefix to the message name indicates its order in the sequence.
- Object1 first sends Object2 the message message1, Object2 in turn sends ObjectN-1 the message message2, and so on.
- Messages that objects send to themselves are indicated as loops (e.g., message message5).



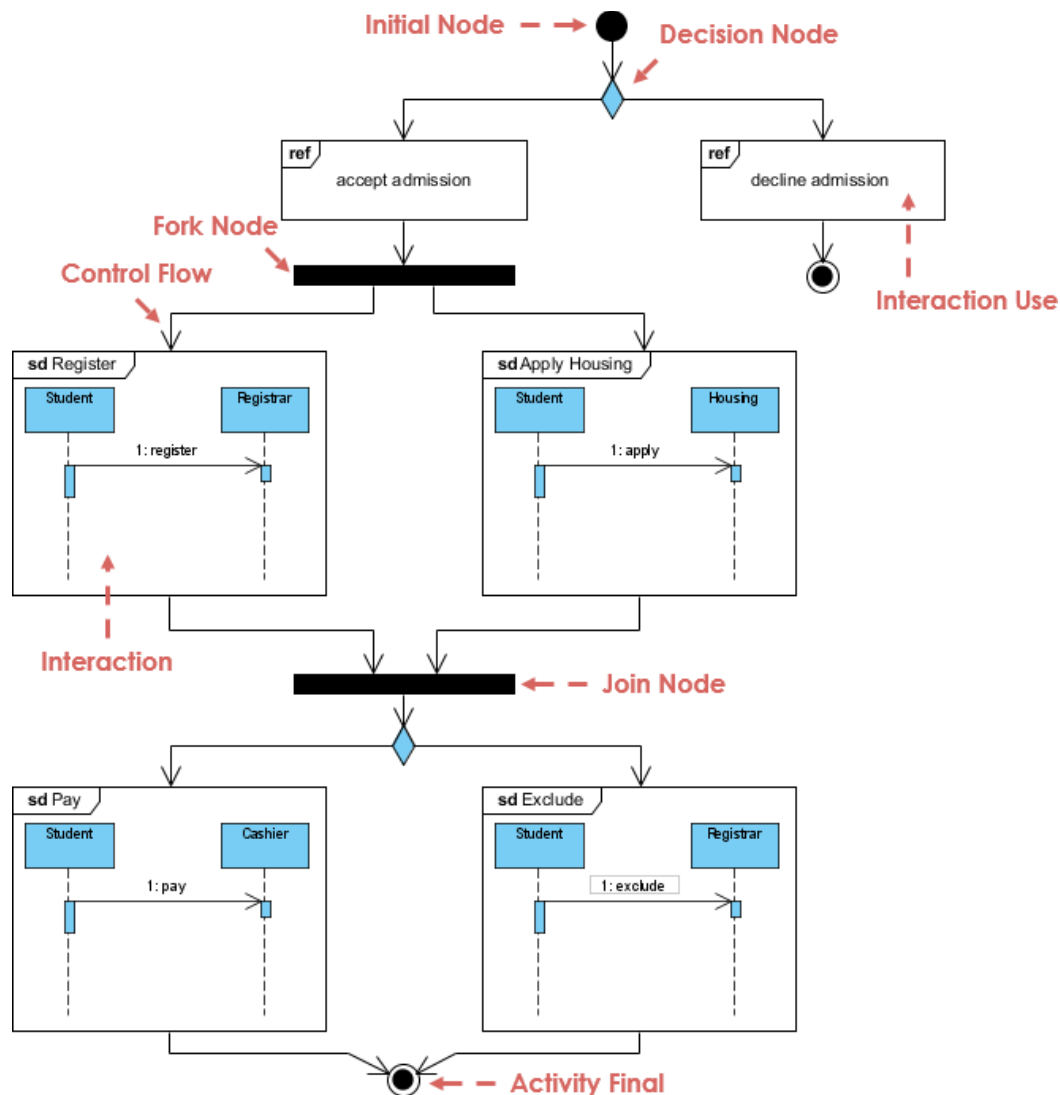
Communication Example - Hotel Reservation



- Each message in a communication diagram has a sequence number.
- The top-level message is numbered 1.
- Messages sent during the same call have the same decimal prefix, but suffixes of 1, 2, etc. according to when they occur.

UML Interaction Overview Diagrams

UML Interaction Overview Diagrams provide a high level of abstraction and interaction model. It is a variant of the Activity Diagram where the nodes are the interactions or interaction occurrences. The Interaction Overview Diagram focuses on the overview of the flow of control of the interactions which can also show the flow of activity between diagrams. In other words, you can link up the "real" diagrams and achieve a high degree of navigability between diagrams inside an Interaction Overview Diagram.



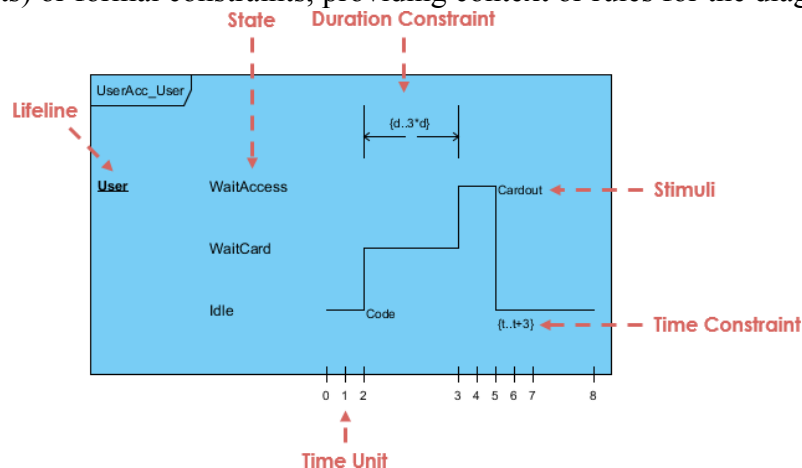
The example above shows a student who has been accepted into a university. First the student must accept or decline admission. After accepting, the student must both register for classes and apply for housing. After both of those are complete, the student must pay the registrar. If payment is not received in time the student is excluded by the registrar.

Node Type	Notation
Interaction An Interaction diagram of any kind may appear inline as an Activity Invocation.	
Interaction Use Large and complex sequence diagrams could be simplified with interaction uses. It is also common to reuse some interaction between several other interactions.	

Timing Diagram:

Timing diagrams are a specialized type of UML interaction diagram. Their main purpose is to model and visualize how the state or condition of one or more objects (lifelines) changes over time, with a strong focus on the timing and sequencing of events or messages that cause these changes.

- **Lifeline:** Represents an individual participant (object, actor, or classifier) in the interaction. Each lifeline is typically drawn as a vertical partition or compartment, labeled with the participant's name.
- **Timeline (Time Axis):** The horizontal axis represents the progression of time, usually increasing from left to right. Tick marks or a timing ruler may be added to indicate specific time intervals.
- **States or Conditions:** Each lifeline shows the possible states or conditions of the object over time. These are depicted as horizontal segments or levels within the lifeline's compartment.
- **Events/Messages:** Arrows or markers indicate events or messages that cause transitions between states. These are placed along the timeline at the point where the state change occurs.
- **Duration Constraints:** Indicate the required or allowed duration that an object remains in a particular state. These are often shown as labeled intervals along the timeline.
- **Time Constraints:** Specify when a particular event or state change must occur, often represented with graphical associations between time intervals and events.
- **Notes and Constraints:** Additional information or restrictions can be attached as notes (comments) or formal constraints, providing context or rules for the diagram elements.



How Timing Diagrams Are Made

- **Identify Participants (Lifelines):** Determine the key objects or components whose state changes need to be tracked.
- **Define States/Conditions:** List the possible states or conditions for each participant.
- **Lay Out the Timeline:** Draw a horizontal time axis, marking relevant time intervals.
- **Map State Changes:** For each lifeline, plot the sequence of state changes along the timeline, indicating when and why each change occurs.
- **Add Events/Messages:** Show the events or messages that trigger state transitions, using arrows or markers at the appropriate time points.
- **Apply Constraints and Notes:** Add any duration or time constraints, as well as explanatory notes, to clarify the timing requirements or behaviors.

Class Diagram:

A UML class diagram is a visual tool that represents the structure of a system by showing its classes, attributes, methods, and the relationships between them. It helps everyone involved in a project—like developers and designers—understand how the system is organized and how its components interact.

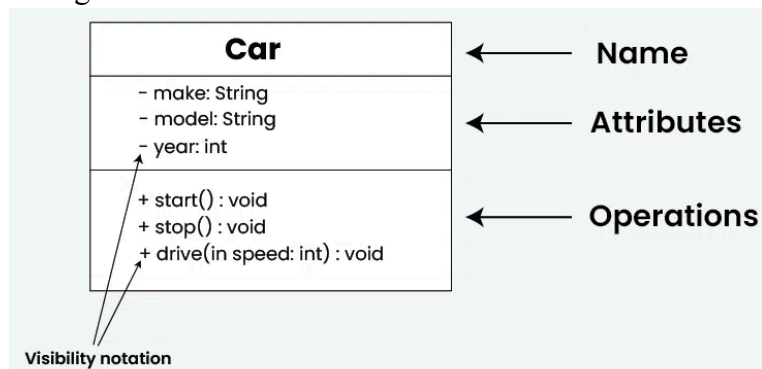
In these diagrams, classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods. Lines connecting classes illustrate associations, showing relationships such as one-to-one or one-to-many.

Class diagrams provide a high-level overview of a system's design, helping to communicate and document the structure of the software. They are a fundamental tool in object-oriented design and play a crucial role in the software development lifecycle.

In object-oriented programming (OOP), a class is a blueprint or template for creating objects. Objects are instances of classes, and each class defines a set of attributes (data members) and methods (functions or procedures) that the objects created from that class will possess. The attributes represent the characteristics or properties of the object, while the methods define the behaviors or actions that the object can perform.

UML Class Notation

class notation is a graphical representation used to depict classes and their relationships in object-oriented modeling.



Class Name: The name of the class is typically written in the top compartment of the class box and is centered and bold.

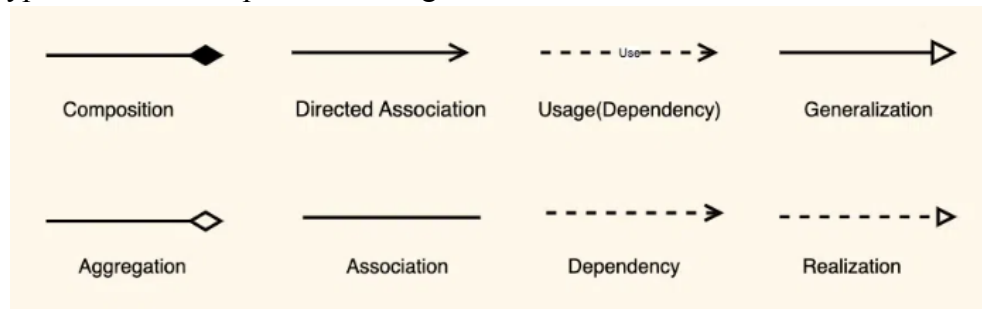
Attributes: Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

Methods: Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

Visibility Notation: Visibility notations indicate the access level of attributes and methods. Common visibility notations include:

- + for public (visible to all classes)
- - for private (visible only within the class)
- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)

Relationships between classes: In class diagrams, relationships between classes describe how classes are connected or interact with each other within a system. There are several types of relationships in object-oriented modeling, each serving a specific purpose. Here are some common types of relationships in class diagrams:



- **Composition:** A strong “has-a” relationship where the part cannot exist without the whole (e.g., a house has rooms).
- **Directed Association:** Indicates a one-way relationship where one class uses or knows about another.
- **Usage (Dependency):** A temporary relationship showing one element uses another for a short time (e.g., as a method parameter).
- **Generalization:** Represents inheritance; a subclass inherits from a superclass.
- **Aggregation:** A weak “has-a” relationship where the part can exist independently of the whole (e.g., a team has players).
- **Association:** A basic relationship showing that two classes are connected in some way.
- **Dependency:** A weaker form of association indicating that a change in one class may affect another.
- **Realization:** Represents an implementation relationship, typically between an interface and a class.

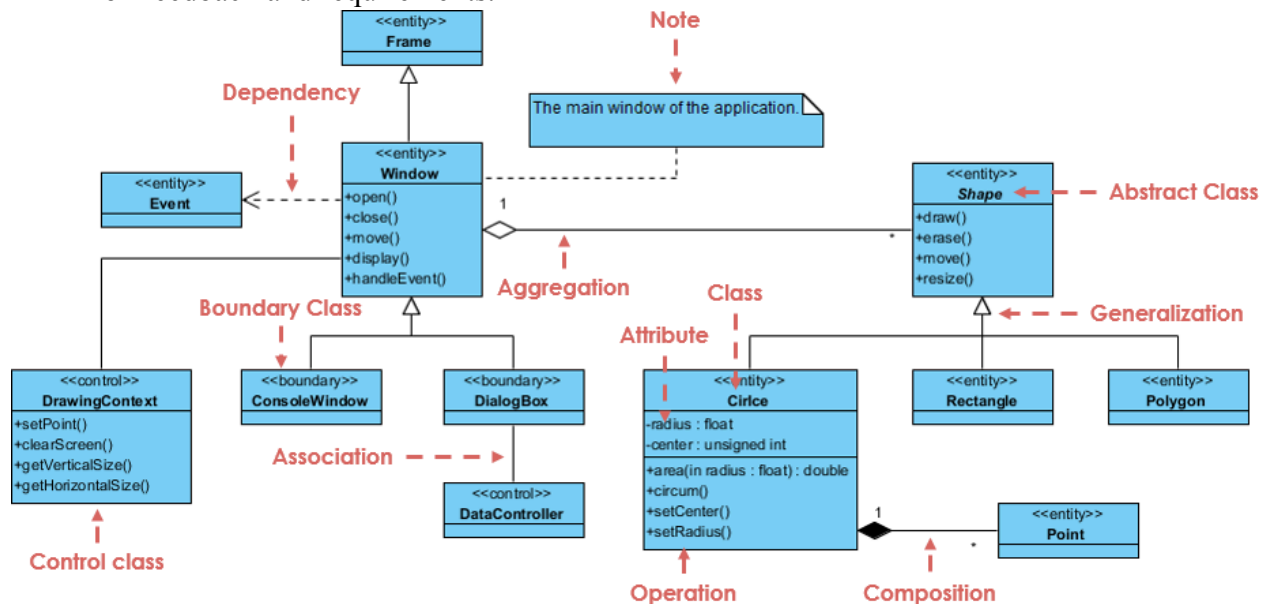
How to draw Class Diagrams ?

Drawing class diagrams involves visualizing the structure of a system, including classes, their attributes, methods, and relationships. Here are the steps to draw class diagrams:

- **Step 1: Identify Classes:** Start by identifying the classes in your system. A class represents a blueprint for objects and should encapsulate related attributes and methods.
- **Step 2: List Attributes and Methods:** For each class, list its attributes (properties, fields) and methods (functions, operations). Include information such as data types and visibility (public, private, protected).
- **Step 3: Identify Relationships:** Determine the relationships between classes. Common relationships include associations, aggregations, compositions, inheritance, and dependencies. Understand the nature and multiplicity of these relationships.
- **Step 4: Create Class Boxes:** Draw a rectangle (class box) for each class identified. Place the class name in the top compartment of the box. Divide the box into compartments for attributes and methods.
- **Step 5: Add Attributes and Methods:** Inside each class box, list the attributes and methods in their respective compartments. Use visibility notations (+ for public, – for private, # for protected, ~ for package/default).
- **Step 6: Draw Relationships:** Draw lines to represent relationships between classes. Use

arrows to indicate the direction of associations or dependencies. Different line types or notations may be used for various relationships.

- **Step 7: Label Relationships:** Label the relationships with multiplicity and role names if needed. Multiplicity indicates the number of instances involved in the relationship, and role names clarify the role of each class in the relationship.
- **Step 8: Review and Refine:** Review your class diagram to ensure it accurately represents the system's structure and relationships. Refine the diagram as needed based on feedback and requirements.



Deployment Diagram:

A Deployment Diagram shows how the software design turns into the actual physical system where the software will run. They show where software components are placed on hardware devices and show how they connect with each other. This diagram helps visualize how the software will operate across different devices

Key elements of a Deployment Diagram

- **Nodes:** These represent the physical hardware entities where software components are deployed, such as servers, workstations, routers, etc.
- **Components:** Represent software modules or artifacts that are deployed onto nodes, including executable files, libraries, databases, and configuration files.
- **Artifacts:** Physical files that are placed on nodes represent the actual implementation of software components. These can include executable files, scripts, databases, and more.
- **Dependencies:** These show the relationships or connections between nodes and components, highlighting communication paths, deployment constraints, and other dependencies.
- **Associations:** Show relationships between nodes and components, signifying that a component is deployed on a particular node, thus mapping software components to physical nodes.
- **Deployment Specification:** This outlines the setup and characteristics of nodes and components, including hardware specifications, software settings, and communication

protocols.

- **Communication Paths:** Represent channels or connections facilitating communication between nodes and components and includes network connections, communication protocols, etc.

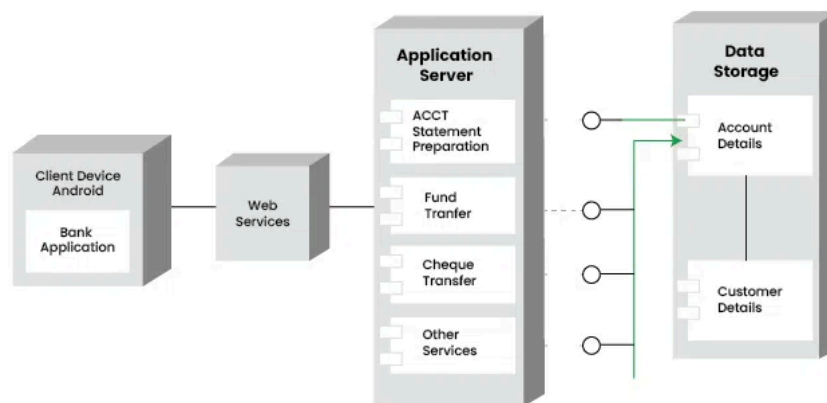
Use Cases of Deployment Diagrams

- Deployment diagrams help plan how software systems will be set up on different devices.
- They help design the hardware needed to support the software. By showing which software parts go where, they help decide what devices and networks are needed.
- Deployment diagrams make sure each part of the software has enough resources, like memory or processing power, to run well.
- They show how different parts of the software depend on each other and on the hardware.
- By seeing how everything is set up, teams can find ways to make the software run faster and smoother.

Steps for creating a Deployment Diagram

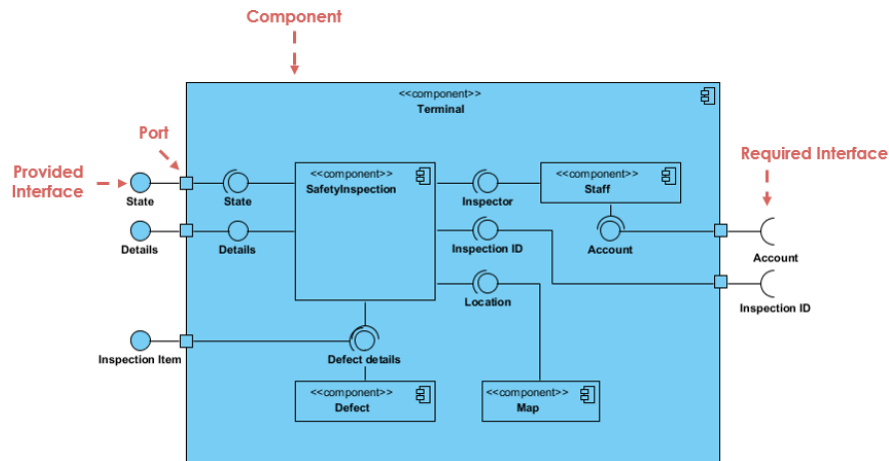
- **Step 1:** Identify Components: List all software parts and hardware devices that will be in the deployment diagram.
- **Step 2:** Understand Relationships: Figure out how these parts connect and work together.
- **Step 3:** Gather Requirements: Collect details about hardware, network setups, and any special rules for deployment.
- **Step 4:** Draw Nodes and Components: Start by drawing the hardware devices (nodes) and software parts (components) using standard symbols roughly at first improvise it and draw the final one.
- **Step 5:** Connect Nodes and Components: Use lines or arrows to show how nodes and components are linked.
- **Step 6:** Add Details: Label everything clearly and include any extra info, like hardware specs or communication protocols.
- **Step 7:** Documentation: Write down any important decisions or assumptions made while creating the diagram.

Deployment Diagram for Mobile Banking Android Services



Component Diagram

A component diagram breaks down the actual system under development into various high levels of functionality. Each component is responsible for one clear aim within the entire system and only interacts with other essential elements on a need-to-know basis.



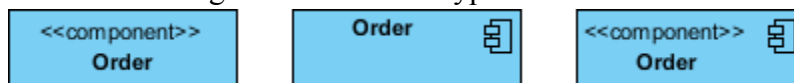
The example above shows the internal components of a larger component:

- The data (account and inspection ID) flows into the component via the port on the right-hand side and is converted into a format the internal components can use. The interfaces on the right are known as required interfaces, which represent the services the component needed in order to carry out its duty.
- The data then passes to and through several other components via various connections before it is output at the ports on the left. Those interfaces on the left are known as the provided interface, which represents the services to be delivered by the exhibiting component.
- It is important to note that the internal components are surrounded by a large 'box' which can be the overall system itself (in which case there would not be a component symbol in the top right corner) or a subsystem or component of the overall system (in this case the 'box' is a component itself).

Basic Concepts of Component Diagram

A component represents a modular part of a system that encapsulates its contents and whose manifestation is replaceable within its environment. In UML 2, a component is drawn as a rectangle with optional compartments stacked vertically. A high-level, abstracted view of a component in UML 2 can be modeled as:

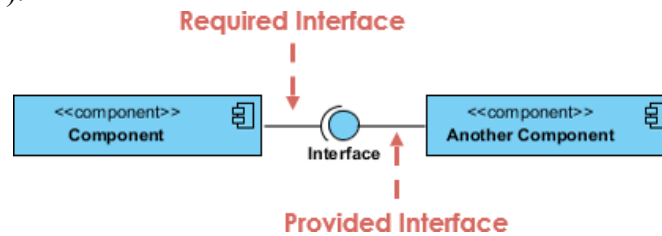
1. A rectangle with the component's name
2. A rectangle with the component icon
3. A rectangle with the stereotype text and/or icon



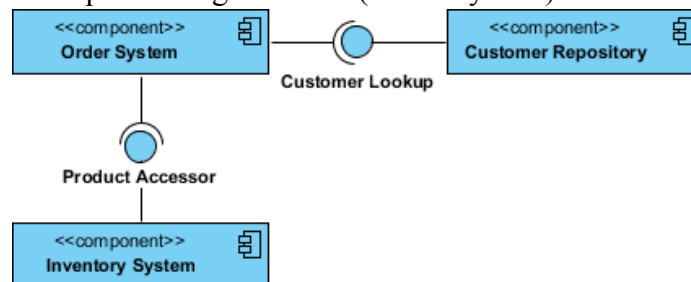
Interface: In the example below shows two type of component interfaces:

Provided interface symbols with a complete circle at their end represent an interface that the component provides - this "lollipop" symbol is shorthand for a realization relationship of an interface classifier.

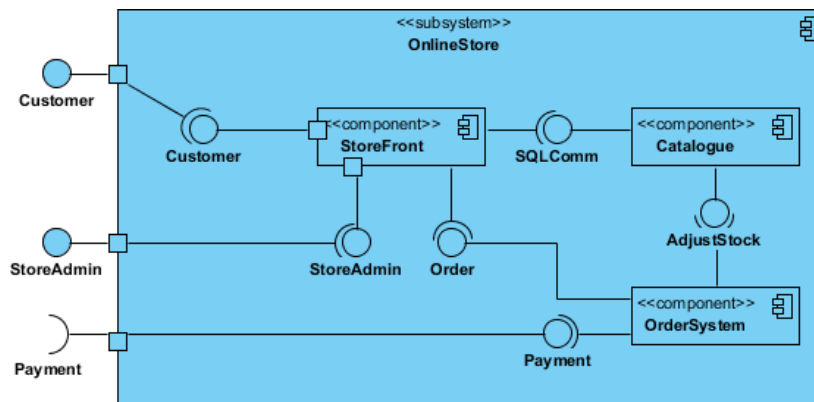
Required Interface symbols with only a half circle at their end (a.k.a. sockets) represent an interface that the component requires (in both cases, the interface's name is placed near the interface symbol itself).



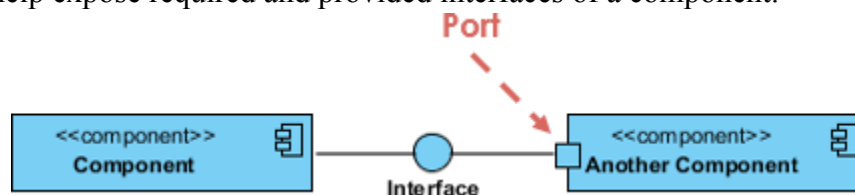
Component Diagram Example - Using Interface (Order System)



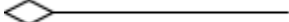
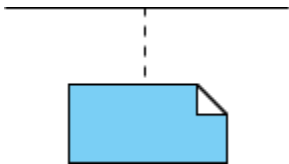
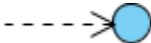
Subsystems: The subsystem classifier is a specialized version of a component classifier. Because of this, the subsystem notation element inherits all the same rules as the component notation element. The only difference is that a subsystem notation element has the keyword of subsystem instead of component.



Port: Ports are represented using a square along the edge of the system or a component. A port is often used to help expose required and provided interfaces of a component.



Relationships: A component diagram is a collection of vertices and arcs and commonly contain components, interfaces and dependency, aggregation, constraint, generalization, association, and realization relationships. It may also contain notes and constraints.

Relationships	Notation
Association: - An association specifies a semantic relationship that can occur between typed instances. - It has at least two ends represented by properties, each of which is connected to the type of the end. More than one end of the association may have the same type.	
Composition: - Composite aggregation is a strong form of aggregation that requires a part instance be included in at most one composite at a time. - If a composite is deleted, all of its parts are normally deleted with it.	
Aggregation A kind of association that has one of its end marks shared as a kind of aggregation, meaning that it has a shared aggregation.	
Constraint A condition or restriction expressed in natural language text or in a machine readable language for the purpose of declaring some of the semantics of an element.	
Dependency - A dependency is a relationship that signifies that a single or a set of model elements requires other model elements for their specification or implementation. - This means that the complete semantics of the dependent elements is either semantically or structurally dependent on the definition of the supplier element(s).	
Links: - A generalization is a taxonomic relationship between a more general classifier and a more specific classifier. - Each instance of the specific classifier is also an indirect instance of the general classifier. - Thus, the specific classifier inherits the features of the more general classifier.	

Data Flow Diagram:

Data flow diagrams are used to graphically represent the flow of data in a business information system. DFD describes the processes that are involved in a system to transfer data from the input to the file storage and reports generation.

Data flow diagrams can be divided into logical and physical. The logical data flow diagram describes flow of data through a system to perform certain functionality of a business. The physical data flow diagram describes the implementation of the logical data flow.

DFD graphically represents the functions, or processes, which capture, manipulate, store, and distribute data between a system and its environment and between components of a system. The visual representation makes it a good communication tool between User and System designer. Structure of DFD allows starting from a broad overview and expanding it to a hierarchy of detailed diagrams. DFD has often been used due to the following reasons:

- Logical information flow of the system
- Determination of physical system construction requirements
- Simplicity of notation
- Establishment of manual and automated systems requirements

DFD Symbols:

There are four basic symbols that are used to represent a data-flow diagram.

Process: A process receives input data and produces output with a different content or form. Processes can be as simple as collecting input data and saving in the database, or it can be complex as producing a report containing monthly sales of all retail stores in the northwest region. Every process has a name that identifies the function it performs. The name consists of a verb, followed by a singular noun.

Example:

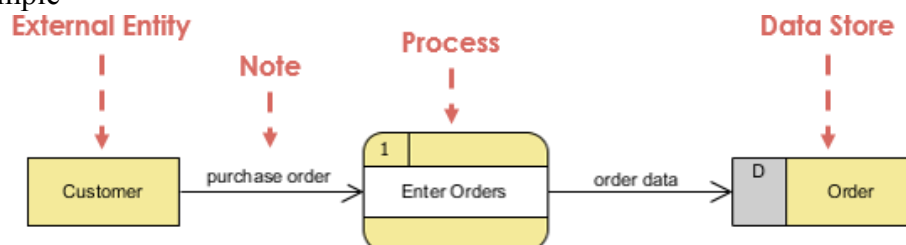
- Apply Payment
- Calculate Commission
- Verify Order

Notation

- A rounded rectangle represents a process
- Processes are given IDs for easy referencing



Process Example

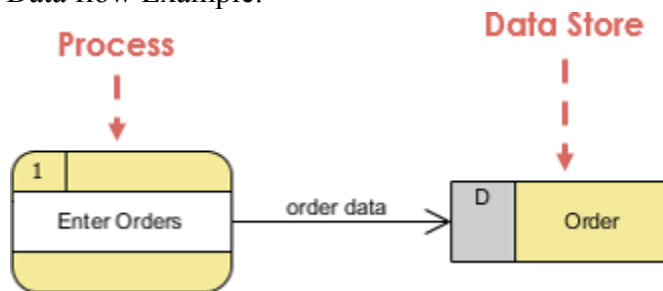


Data Flow: A data-flow is a path for data to move from one part of the information system to another. A data-flow may represent a single data element such as the Customer ID or it can represent a set of data elements (or a data structure).

Example:

- Customer_info (LastName, FirstName, SS#, Tel #, etc.)
- Order_info (OrderId, Item#, OrderDate, CustomerID, etc.).

Data flow Example:



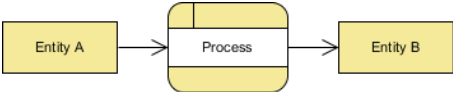
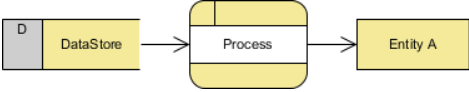
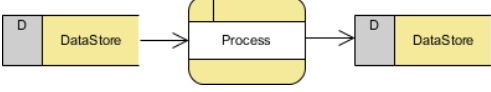
Notation

- Straight lines with incoming arrows are input data flow
- Straight lines with outgoing arrows are output data flows

Note that: Because every process changes data from one form into another, at least one data-flow must enter and one data-flow must exit each process symbol.

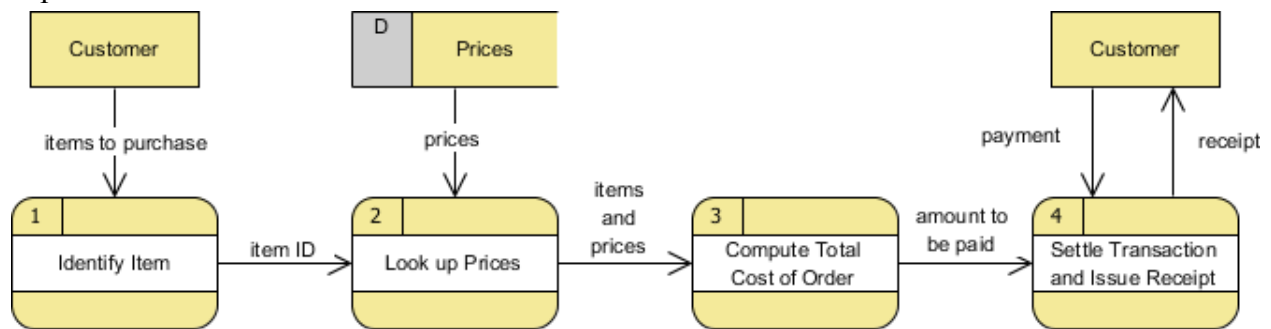
Rule of Data Flow

One of the rules for developing DFD is that all flow must begin with and end at a processing step. This is quite logical, because data can't transform on its own without being processed. By using the thumb rule, it is quite easy to identify the illegal data flows and correct them in a DFD.

Right	Description
	An entity cannot provide data to another entity without some processing occurred.
	Data cannot move directly from an entity to a data store without being processed.
	Data cannot move directly from a data store without being processed.
	Data cannot move directly from one data store to another without being processed.

Logical DFD Example - Grocery Store

The logical DFD illustrates the processes involved without going into detail about the physical implementation of activities.



Physical DFD Example - Grocery Store

- The physical DFD shows that a barcode-the UPC PRICE code found on most grocery store items is used
- In addition, the physical DFD mentions manual processes such as scanning, explains that a temporary file is used to keep a subtotal of items
- The PAYMENT could be made by CASH, CHECK, or DEBIT CARD
- Finally, it refers to the receipt by its name, CASH REGISTER RECEIPT

